

Logical Decoding and Change Data Capture in PostgreSQL

Table of Contents

- 1. [Learning Objectives](#)
- 2. [What is Logical Decoding?](#)
- 3. [WAL and Logical Replication Architecture](#)
- 4. [Replication Slots](#)
- 5. [Output Plugins](#)
- 6. [pg_logical Extension](#)
- 7. [Logical Replication \(Built-in\)](#)
- 8. [Debezium Integration](#)
- 9. [CDC Patterns and Use Cases](#)
- 10. [Production Monitoring Queries](#)
- 11. [Common Mistakes](#)
- 12. [Best Practices](#)
- 13. [Performance Considerations](#)
- 14. [Interview Questions & Answers](#)
- 15. [Exercises and Solutions](#)
- 16. [Cross-References](#)

Learning Objectives

By the end of this module you will be able to:

- Explain the WAL-based logical decoding pipeline end-to-end
- Create, monitor, and safely manage replication slots
- Use `test_decoding` and `wal2json` output plugins for CDC
- Set up built-in logical replication for table-level data sync
- Understand how Debezium reads PostgreSQL change events
- Implement CDC patterns for event sourcing, search sync, and cache invalidation
- Diagnose and resolve replication slot lag issues

What is Logical Decoding?

Logical decoding transforms PostgreSQL's low-level Write-Ahead Log (WAL) into a stream of logical data changes (INSERT, UPDATE, DELETE) at the row level, independent of the physical storage format.

Without Logical Decoding:

With Logical Decoding:

WAL Record:

rm: heap

info: HOT_UPDATE

blkno: 42

offnum: 5

lsn: 0/15E0A90

—decode→

Decoded Output:

BEGIN 500

table public.users:

UPDATE: id=1 old:(name='Bob')

new:(name='Alice')

COMMIT 500

(opaque binary)

(human-readable row changes)

Prerequisites

```
-- Check current WAL level
SHOW wal_level;
-- Must be 'logical' for logical decoding

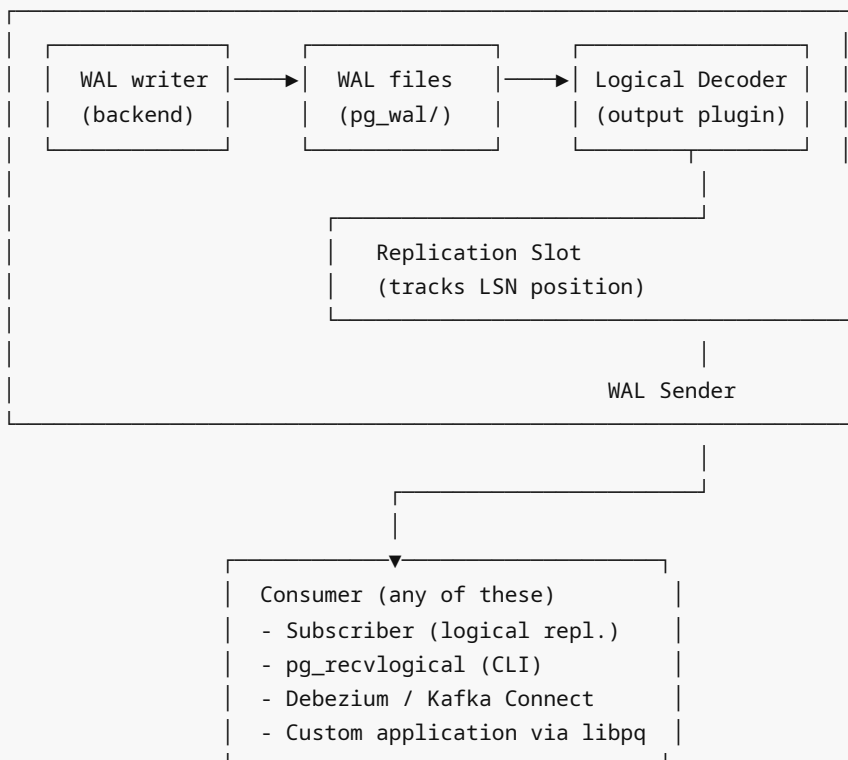
-- Change in postgresql.conf:
-- wal_level = logical
-- max_replication_slots = 10    (default 10)
-- max_wal_senders = 10          (default 10)
-- wal_sender_timeout = 60s

-- Then restart PostgreSQL
-- sudo systemctl restart postgresql

-- Verify
SHOW wal_level;           -- logical
SHOW max_replication_slots; -- 10
```

WAL and Logical Replication Architecture

PostgreSQL Server:



Replication Slots

A replication slot tracks the WAL position for a consumer. PostgreSQL guarantees WAL is retained until all slots have consumed it.

```
-- Create a logical replication slot
SELECT pg_create_logical_replication_slot('my_slot', 'test_decoding');
-- Returns: (slot_name, lsn)

-- Create slot with wal2json plugin
SELECT pg_create_logical_replication_slot('json_slot', 'wal2json');

-- List all replication slots
SELECT
    slot_name,
    plugin,
    slot_type,
    active,
    restart_lsn,
    confirmed_flush_lsn,
    pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS lag_bytes,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS lag_pretty
FROM pg_replication_slots;

-- Peek at changes (does NOT advance the slot)
SELECT lsn, xid, data
FROM pg_logical_slot_peek_changes('my_slot', NULL, NULL);

-- Get changes (ADVANCES the slot – data consumed)
SELECT lsn, xid, data
FROM pg_logical_slot_get_changes('my_slot', NULL, NULL);

-- Advance slot to current position (discard unread changes)
SELECT pg_replication_slot_advance('my_slot', pg_current_wal_lsn());

-- Drop slot (important: idle slots retain WAL indefinitely!)
SELECT pg_drop_replication_slot('my_slot');
```

Danger: Unused replication slots

Warning: Inactive Slot Risk

```
Active WAL Position: 0/50000000
Slot restart_lsn:    0/10000000 ← slot stuck here
                    |
                    └─ PostgreSQL MUST retain all WAL from
                        0/10000000 onwards
                        = 1GB of WAL cannot be recycled
```

If disk fills: PostgreSQL will STOP accepting writes!

```
-- Monitor slot lag (CRITICAL production query – run every minute)
SELECT
    slot_name,
    active,
    pg_size_pretty(pg_wal_lsn_diff(
        pg_current_wal_lsn(),
        COALESCE(confirmed_flush_lsn, restart_lsn)
    )) AS lag
FROM pg_replication_slots
WHERE pg_wal_lsn_diff(
    pg_current_wal_lsn(),
    COALESCE(confirmed_flush_lsn, restart_lsn)
) > 1073741824; -- alert if lag > 1GB
```

Output Plugins

test_decoding (built-in)

```
-- Simple text-based output for testing
SELECT pg_create_logical_replication_slot('test_slot', 'test_decoding');

-- Generate some changes
BEGIN;
INSERT INTO users (id, name) VALUES (1, 'Alice');
UPDATE users SET name = 'Alicia' WHERE id = 1;
DELETE FROM users WHERE id = 1;
COMMIT;

-- Read decoded changes
SELECT * FROM pg_logical_slot_get_changes('test_slot', NULL, NULL);
/*
 lsn          | xid | data
-----+-----+-----
0/15E0A90 | 500 | BEGIN 500
0/15E0B00 | 500 | table public.users: INSERT: id[int4]:1 name[text]:'Alice'
0/15E0B80 | 500 | table public.users: UPDATE: id[int4]:1 name[text]:'Alicia'
0/15E0C00 | 500 | table public.users: DELETE: id[int4]:1
0/15E0C80 | 500 | COMMIT 500
*/
```

wal2json (popular external plugin)

```
-- Requires: wal2json package installed
-- Ubuntu: sudo apt install postgresql-15-wal2json

SELECT pg_create_logical_replication_slot('json_slot', 'wal2json');

SELECT data FROM pg_logical_slot_get_changes('json_slot', NULL, NULL,
```

```

    'pretty-print', '1',
    'include-timestamp', '1',
    'include-type-oids', '0'
);
/*
{
  "change": [
    {
      "kind": "insert",
      "schema": "public",
      "table": "users",
      "columnnames": ["id", "name", "email"],
      "columnvalues": [1, "Alice", "alice@example.com"]
    },
    {
      "kind": "update",
      "schema": "public",
      "table": "users",
      "columnnames": ["id", "name", "email"],
      "columnvalues": [1, "Alicia", "alice@example.com"],
      "oldkeys": {"keynames": ["id"], "keyvalues": [1]}
    }
  ]
}
*/

```

pgoutput (PostgreSQL 10+ built-in for logical replication)

```

-- Used by the built-in logical replication system
-- Configure publication for pgoutput
CREATE PUBLICATION my_pub FOR TABLE users, orders;

-- pgoutput is the only plugin needed for native logical replication

```

pg_logical Extension

```

-- Install pg_logical for advanced logical replication scenarios
CREATE EXTENSION IF NOT EXISTS pglogical;

-- On provider (source) node:
SELECT pglogical.create_node(
  node_name => 'provider1',
  dsn => 'host=source_host dbname=mydb'
);

-- Add table to replication set
SELECT pglogical.replication_set_add_table(
  set_name    => 'default',
  relation    => 'public.users',

```

```

        synchronize_data => true
    );

-- On subscriber (target) node:
SELECT pglogical.create_node(
    node_name => 'subscriber1',
    dsn => 'host=this_host dbname=mydb'
);

SELECT pglogical.create_subscription(
    subscription_name => 'my_subscription',
    provider_dsn      => 'host=source_host dbname=mydb',
    replication_sets  => ARRAY['default'],
    synchronize_structure => true,
    synchronize_data  => true
);

```

Logical Replication (Built-in)

PostgreSQL 10+ includes native logical replication.

```

-- === ON SOURCE DATABASE ===

-- Grant replication privilege
CREATE ROLE replication_user LOGIN REPLICATION PASSWORD 'secret';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO replication_user;

-- Create publication
CREATE PUBLICATION orders_pub FOR TABLE orders, order_items;
CREATE PUBLICATION all_tables_pub FOR ALL TABLES;
CREATE PUBLICATION filtered_pub FOR TABLE orders WHERE (status = 'completed');

-- List publications
SELECT * FROM pg_publication;
SELECT * FROM pg_publication_tables;

-- === ON DESTINATION DATABASE ===

-- Create subscription (pulls from source)
CREATE SUBSCRIPTION orders_sub
    CONNECTION 'host=source_db port=5432 dbname=mydb user=replication_user
password=secret'
    PUBLICATION orders_pub
    WITH (copy_data = true, synchronous_commit = off);

-- List subscriptions
SELECT * FROM pg_subscription;
SELECT * FROM pg_stat_subscription;

-- Refresh subscription after source schema changes

```

```

ALTER SUBSCRIPTION orders_sub REFRESH PUBLICATION;

-- Disable/enable
ALTER SUBSCRIPTION orders_sub DISABLE;
ALTER SUBSCRIPTION orders_sub ENABLE;

-- Drop
DROP SUBSCRIPTION orders_sub;

-- === MONITORING ===
-- Check replication lag on source
SELECT
    client_addr,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;

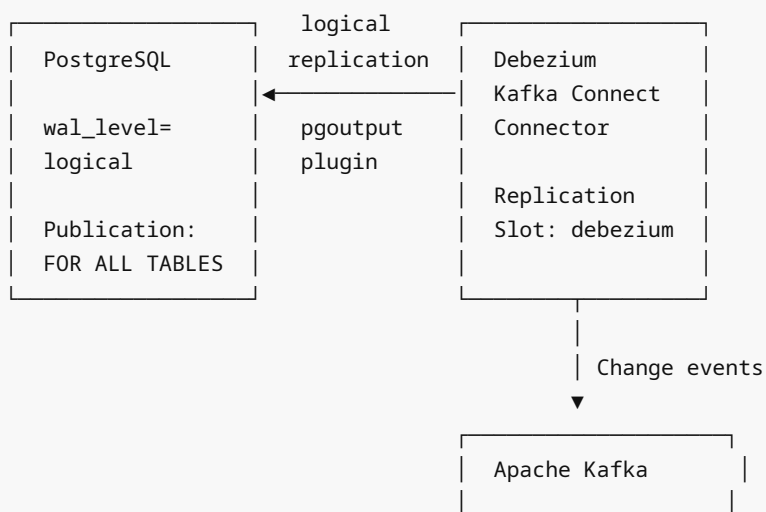
-- Check subscription status
SELECT
    subname,
    received_lsn,
    latest_end_lsn,
    latest_end_time
FROM pg_stat_subscription;

```

Debezium Integration

Debezium is an open-source CDC platform that reads PostgreSQL logical replication and publishes change events to Kafka.

Debezium + PostgreSQL Architecture:



topic: db.users
topic: db.orders

PostgreSQL configuration for Debezium

```
-- postgresql.conf changes:
-- wal_level = logical
-- max_replication_slots = 4
-- max_wal_senders = 4

-- Create replication user for Debezium
CREATE ROLE debezium LOGIN REPLICATION PASSWORD 'dbz_password';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO debezium;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO debezium;

-- Create publication (Debezium creates slot automatically)
CREATE PUBLICATION dbz_publication FOR ALL TABLES;

-- Verify publication
SELECT * FROM pg_publication;
```

Debezium connector config (Kafka Connect)

```
{
  "name": "postgres-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "postgres-host",
    "database.port": "5432",
    "database.user": "debezium",
    "database.password": "dbz_password",
    "database.dbname": "mydb",
    "database.server.name": "myserver",
    "plugin.name": "pgoutput",
    "publication.name": "dbz_publication",
    "slot.name": "debezium_slot",
    "table.include.list": "public.orders,public.users",
    "heartbeat.interval.ms": "5000",
    "tombstones.on.delete": "true",
    "decimal.handling.mode": "double"
  }
}
```

Debezium event format

```
{
  "op": "u",
  "ts_ms": 1704067200000,
```

```

    "before": {
      "id": 1,
      "status": "pending",
      "total": 99.99
    },
    "after": {
      "id": 1,
      "status": "completed",
      "total": 99.99
    },
    "source": {
      "db": "mydb",
      "schema": "public",
      "table": "orders",
      "lsn": 24023960,
      "txId": 500
    }
  }
}

```

CDC Patterns and Use Cases

Pattern 1: Elasticsearch Sync

```

-- Use Debezium to sync PostgreSQL → Elasticsearch
-- Configure: table.include.list = public.products
-- Kafka consumer reads events and calls Elasticsearch API

-- Ensure REPLICA IDENTITY includes all needed fields
ALTER TABLE products REPLICA IDENTITY FULL;
-- Default REPLICA IDENTITY DEFAULT only includes primary key in DELETE events
-- FULL includes all columns in DELETE old-row image

```

Pattern 2: Cache Invalidation

```

-- Application reads from Redis cache
-- Debezium (or custom slot consumer) invalidates cache on change

-- Custom consumer using pg_logical_slot_get_changes:
CREATE OR REPLACE FUNCTION consume_changes_for_cache()
RETURNS void LANGUAGE plpgsql AS $$
DECLARE
  r RECORD;
BEGIN
  FOR r IN
    SELECT data
    FROM pg_logical_slot_get_changes('cache_slot', NULL, 100,
    'include-types', '0')
  LOOP
    -- Parse r.data and extract affected table + primary key

```

```
-- Call Redis DEL/INVALIDATE via pg_http or notify application
RAISE NOTICE 'Change: %', r.data;
END LOOP;
END;
$$;
```

Pattern 3: Audit Log via CDC

```
-- Create a slot that feeds an audit log table
CREATE TABLE change_log (
  id          BIGSERIAL PRIMARY KEY,
  lsn         PG_LSN,
  xid         INT,
  table_name  TEXT,
  operation   TEXT, -- INSERT/UPDATE/DELETE
  row_data    JSONB,
  captured_at TIMESTAMPTZ DEFAULT now()
);

-- Consumer process reads from slot and writes to change_log
-- This is more reliable than trigger-based audit (survives DDL changes)
```

Pattern 4: Event Sourcing

```
-- Use logical replication as the event bus
-- Events table is source of truth
CREATE TABLE domain_events (
  id          BIGSERIAL PRIMARY KEY,
  event_type  TEXT NOT NULL,
  aggregate_id BIGINT NOT NULL,
  payload     JSONB NOT NULL,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Logical replication publishes all inserts to Kafka
-- Consumers replay events to build read models
CREATE PUBLICATION events_pub FOR TABLE domain_events;
```

Production Monitoring Queries

```
-- 1. CRITICAL: Check all replication slot lag
SELECT
  slot_name,
  plugin,
  slot_type,
  active,
  active_pid,
```

```

pg_size_pretty(
    pg_wal_lsn_diff(pg_current_wal_lsn(),
        COALESCE(confirmed_flush_lsn, restart_lsn))
    ) AS lag,
pg_wal_lsn_diff(pg_current_wal_lsn(),
    COALESCE(confirmed_flush_lsn, restart_lsn)) AS lag_bytes
FROM pg_replication_slots
ORDER BY lag_bytes DESC;

-- 2. WAL disk usage by slots
SELECT
    slot_name,
    restart_lsn,
    pg_current_wal_lsn(),
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
wal_retained
FROM pg_replication_slots;

-- 3. Active WAL senders (logical and physical)
SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    sent_lsn,
    write_lsn,
    replay_lsn,
    pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes,
    sync_state
FROM pg_stat_replication;

-- 4. Subscription status (on subscriber)
SELECT
    subname,
    pid,
    received_lsn,
    last_msg_send_time,
    last_msg_receipt_time,
    latest_end_lsn,
    latest_end_time
FROM pg_stat_subscription;

-- 5. How many WAL files are being retained?
SELECT COUNT(*) AS wal_file_count,
    pg_size_pretty(COUNT(*) * 16 * 1024 * 1024) AS approx_wal_size
FROM pg_ls_waldir()
WHERE name ~ '^[0-9A-F]{24}$';

-- 6. Check for stale/inactive slots
SELECT slot_name, active, restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots

```

```
WHERE active = false;
-- Non-empty result = WARNING: these slots are retaining WAL

-- 7. Logical replication worker status
SELECT *
FROM pg_stat_replication_slots; -- PostgreSQL 14+
```

Common Mistakes

1. **Leaving inactive replication slots running** — An inactive slot retains all WAL from its restart_lsn. This is the #1 cause of disk exhaustion in PostgreSQL. Drop unused slots immediately.
 2. **Not monitoring slot lag** — A slot that falls behind by gigabytes of WAL can cause disk full events. Set up an alert when lag > 500MB.
 3. **Using test_decoding in production** — test_decoding is for development only. Use pgoutput or wal2json in production.
 4. **Forgetting REPLICATION IDENTITY for DELETE** — By default, DELETes only include the primary key in the WAL record. ALTER TABLE t REPLICATION IDENTITY FULL includes all columns, needed for Debezium and some CDC patterns.
 5. **Missing max_wal_senders and max_replication_slots** — These must be set before any slots are created. Increasing them requires a server restart.
 6. **Not granting REPLICATION privilege** — The replication user needs LOGIN REPLICATION role attributes and SELECT on all replicated tables.
-

Best Practices

1. **Set max_replication_slots conservatively** — Each slot retains WAL even if inactive. Set to actual expected consumers + 2 spare.
 2. **Always drop unused slots** — Automate cleanup: if a slot is inactive for > 24 hours, alert and auto-drop.
 3. **Use heartbeats in Debezium** — heartbeat.interval.ms prevents slot lag from growing on low-traffic tables by periodically updating the slot's LSN.
 4. **Use REPLICATION IDENTITY FULL judiciously** — It increases WAL volume for every UPDATE (must log all old values). Only use for tables where full-row audit is required.
 5. **Monitor pg_current_wal_lsn() - confirmed_flush_lsn** — This is the true lag. Alert at 500MB, page at 2GB.
-

Performance Considerations

```
-- Check WAL generation rate
SELECT pg_wal_lsn_diff(pg_current_wal_lsn(), '0/0') / 1024 / 1024 AS total_wal_mb;
```

```
-- Estimate WAL generation per hour
WITH a AS (SELECT pg_current_wal_lsn() AS lsn1, clock_timestamp() AS t1),
      b AS (SELECT pg_current_wal_lsn() AS lsn2, clock_timestamp() AS t2,
                (SELECT lsn1 FROM a) AS lsn1, (SELECT t1 FROM a) AS t1)
SELECT
  pg_size_pretty(pg_wal_lsn_diff(lsn2, lsn1)) AS wal_since_sample,
  EXTRACT(EPOCH FROM (t2 - t1)) AS seconds_elapsed
FROM b;

-- Logical decoding overhead: ~5-15% CPU for high-write workloads
-- Monitor: pg_stat_activity for walsender processes
SELECT pid, state, wait_event_type, wait_event, backend_type
FROM pg_stat_activity
WHERE backend_type IN ('walsender', 'logical replication worker');
```

Interview Questions & Answers

Q1: What is the difference between logical and physical replication in PostgreSQL?

A: Physical replication copies the raw WAL bytes (binary) — an exact replica of the file system. The replica must be the same PostgreSQL version and OS architecture. Logical replication decodes WAL into SQL-level changes (INSERT/UPDATE/DELETE). It works across versions and architectures, supports table-level granularity, and can filter or transform data. Physical replication is used for HA (high availability); logical for cross-version upgrades, partial replication, and CDC.

Q2: What happens if a replication slot is never consumed?

A: PostgreSQL retains all WAL from the slot's restart_lsn onwards. The WAL directory grows unboundedly. When the disk fills, PostgreSQL can no longer write WAL and stops accepting write transactions — a complete outage. This is one of the most dangerous PostgreSQL operational issues. Monitor slot lag continuously and drop unused slots immediately.

Q3: What is REPLICA IDENTITY and why does it matter for CDC?

A: REPLICA IDENTITY controls what old-row data is included in WAL records for UPDATE and DELETE operations. `DEFAULT` includes only the primary key. `FULL` includes all column values. `NOTHING` includes no old values. `USING INDEX idx` includes the indexed columns. For CDC systems like Debezium that need the old values of changed columns (to identify which record changed without a primary key, or to detect what changed), `REPLICA IDENTITY FULL` is required.

Q4: Explain the role of the `confirmed_flush_lsn` vs `restart_lsn` in a replication slot.

A: `restart_lsn` is the earliest WAL position that must be retained for this slot — the slot can't decode WAL older than this. `confirmed_flush_lsn` is the latest position the consumer has successfully processed and acknowledged (flushed). `pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)` gives the true consumer lag. `restart_lsn` may be slightly behind `confirmed_flush_lsn` to allow for consumer restarts.

Q5: How does Debezium handle the initial snapshot?

A: When a Debezium connector starts for the first time, it performs an initial consistent snapshot: it takes an ACCESS SHARE lock on tables, reads all existing rows, and publishes them as INSERT events. After the

snapshot, it switches to reading from the replication slot for ongoing changes. This ensures the consumer gets a complete picture of the current data before receiving incremental changes.

Q6: What is `wal_level = logical` and what overhead does it add?

A: `wal_level = logical` causes PostgreSQL to include additional information in WAL records needed for logical decoding — specifically, the old row values for UPDATE/DELETE operations. This increases WAL volume by roughly 5-20% for UPDATE-heavy workloads compared to `replica` level. It also requires a server restart to enable. The overhead is generally acceptable for production given the benefit of logical decoding capability.

Q7: How would you safely drop a Debezium replication slot when Debezium is down?

A: First verify the slot is inactive: `SELECT active FROM pg_replication_slots WHERE slot_name = 'debezium_slot'`. If inactive, note the lag with `pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)`. If dropping is intentional (e.g., decommissioning the connector), run `SELECT pg_drop_replication_slot('debezium_slot')`. When Debezium restarts without the slot, it will perform a new initial snapshot. Plan for the snapshot to re-read all data.

Q8: What is the difference between a publication and a replication slot?

A: A publication defines WHAT is replicated — it's a named set of tables (and optionally filtered rows) with a specified set of operations (INSERT/UPDATE/DELETE). A replication slot tracks WHERE a specific consumer is in the WAL stream — it maintains the consumer's position and ensures WAL is retained. A publication is referenced by multiple subscribers; each subscriber has its own replication slot. You can have multiple subscribers reading the same publication, each with their own slot.

Exercises and Solutions

Exercise 1

Create a replication slot using `test_decoding`, perform INSERT/UPDATE/DELETE operations, decode the changes, then clean up.

Solution:

```
-- Setup
CREATE TABLE cdc_test (id SERIAL PRIMARY KEY, value TEXT);
SELECT pg_create_logical_replication_slot('test_cdc', 'test_decoding');

-- Make changes
BEGIN;
INSERT INTO cdc_test VALUES (1, 'hello');
UPDATE cdc_test SET value = 'world' WHERE id = 1;
DELETE FROM cdc_test WHERE id = 1;
COMMIT;

-- Decode
SELECT lsn, xid, data
FROM pg_logical_slot_get_changes('test_cdc', NULL, NULL);

-- Cleanup
```

```
SELECT pg_drop_replication_slot('test_cdc');
DROP TABLE cdc_test;
```

Exercise 2

Write a monitoring query that alerts if any replication slot has more than 500MB of unconfirmed WAL, showing the slot name, lag, and whether it's active.

Solution:

```
SELECT
    slot_name,
    plugin,
    active,
    active_pid,
    pg_size_pretty(
        pg_wal_lsn_diff(
            pg_current_wal_lsn(),
            COALESCE(confirmed_flush_lsn, restart_lsn)
        )
    ) AS lag,
    CASE
        WHEN active = false THEN 'CRITICAL: Slot inactive and retaining WAL!'
        WHEN pg_wal_lsn_diff(
            pg_current_wal_lsn(),
            COALESCE(confirmed_flush_lsn, restart_lsn)
        ) > 524288000 THEN 'WARNING: Slot lag > 500MB'
        ELSE 'OK'
    END AS status
FROM pg_replication_slots
WHERE pg_wal_lsn_diff(
    pg_current_wal_lsn(),
    COALESCE(confirmed_flush_lsn, restart_lsn)
) > 524288000
OR active = false
ORDER BY pg_wal_lsn_diff(
    pg_current_wal_lsn(),
    COALESCE(confirmed_flush_lsn, restart_lsn)
) DESC;
```

Cross-References

- **10_postgresql_internals** — WAL structure and format
- **13_replication_ha** — Physical vs logical replication, streaming replication setup
- **09_stored_procedures_functions.md** — Trigger-based CDC alternative
- **07_incident_response.md** (12_Production) — Replication slot runbook
- **06_maintenance_procedures.md** (12_Production) — Slot monitoring in maintenance
- **Debezium docs** — <https://debezium.io/documentation/reference/stable/connectors/postgresql.html>